

Ejercicios-memoria-cache.pdf



JDTadasGOT



Estructura de Computadores



2º Grado en Ingeniería Informática



**Facultad de Informática
Universidad Complutense de Madrid**

PROBLEMAS DE ESTRUCTURA DE COMPUTADORES

MEMORIA

Memoria Cache - Problemas básicos

1.- En un computador con caches separadas de datos e instrucciones, de 32KB cada una y bloques de 256 bytes, se quiere ejecutar el siguiente código C, que realiza la operación matricial $C = A+B$:

```
#define N 128
int A[N][N];
int B[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = A[i][j] + B[i][j];
```

Asumiendo que la matriz **A** se ubica en la dirección **0x0C000000**, que B y C se almacenan consecutivamente a continuación de A, y que en la traducción a ensamblador del código anterior la variable *i* se ubica en un registro y que las direcciones de comienzo de los arrays están también en registro, se pide:

- Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es de emplazamiento directo sin asignación en escritura.
- Repetir el cálculo para una cache asociativa por conjuntos, de 2 vías, con política de reemplazamiento LRU, en los siguientes casos:
 - sin asignación en escritura.
 - con asignación en escritura.

En cada caso indicar si el rendimiento mejoraría o empeoraría y en qué proporción.

- Un programador sugiere la siguiente modificación de código para una cache asociativa por conjuntos de 2 vías con asignación en escritura:

```
#define N 128
struct {
    int A;
    int B;
} AB[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = AB[i][j].A + AB[i][j].B;
```

Asumiendo que el array AB se ubica en la dirección 0x0C000000 y que el array C se ubica a continuación, calcule el número de fallos en lectura y escritura que se producirían, y compárelo con el resultado del apartado b para la misma configuración de cache.

- ¿Puede obtenerse un resultado similar al del apartado c con el código original y sin aumentar el tamaño de la cache? Razone la respuesta.

2.- Sea un computador con una memoria principal de 64K bytes, direccionable en bytes, al que se dota de una memoria cache de 2K bytes, con líneas de 256 bytes, y prebúsqueda bajo fallo (en caso de fallo se trae el bloque que lo provoca y el siguiente).

Sea la secuencia de acceso a memoria principal dada por las siguientes direcciones: 0x3345, 0x14BF, 0x4021, 0x1184, 0x55AB, 0x1284 y 0x41F1.

Muestre la evolución de la caché de etiquetas indicando los fallos y las prebúsquedas que se producen en los siguientes casos:

- Una organización directa
- Una organización asociativa por conjuntos de 2 vías, con algoritmo de reemplazo LRU
- Una organización directa y un buffer de 512 bytes donde se almacena inicialmente el bloque prebuscado hasta que se le referencia. El buffer actúa con emplazamiento asociativo y política FIFO.

3.- Considere un computador con una memoria principal de 4MB, direccionable por bytes, al que se dota de una memoria cache de 4KB, con líneas de 512B, y cache de víctima de 1024B. La memoria cache tiene dos vías mientras que la cache víctima es completamente asociativa. Para ambas el algoritmo de reemplazamiento es FIFO. Originalmente en las dos memorias se encuentran los bloques que aparecen en las siguientes tablas:

Conjunto	Etiqueta	FIFO	Etiqueta/ Conjunto de MP	FIFO
0	668	0	0A80	0
0	008	1	3FFF	1
1	297	0		
1	368	1		
2	5FF	0		
2	668	1		
3	297	1		
3	200	0		

Sea la secuencia de acceso a memoria principal dada por las siguientes direcciones: 0x334500, 0x14BF00, 0x150084, 0x004021, 0x0540AB y 0x0041F1.
Muestre la evolución de la caché de etiquetas indicando los fallos y las transferencias entre cache de víctima y cache.

4.- Considere un computador con direcciones de 32 bits, al que se dota de una memoria cache de 128 bytes, con bloques de 16 bytes y emplazamiento directo, sin asignación en escritura (No write-allocate). Suponga que la variable a está en el banco de registros y no en cache.
Se quiere ejecutar el siguiente código:

```
int nota[128];    // nota[0] está en la dirección 0x00000000
int media[128];  // media [0] está a continuación de nota[127]

for (i=0;i<128;i++) {
    if (i>7 && i<64) {
        nota[i] = media[i]/2;
    }
    else {
        a = nota[i]*media[i]
    }
}
```

- Indique cuántos fallos de cache se producen.
- Proponga dos optimizaciones de código (independientes) que mejoren el resultado, indicando cuántos fallos se producen en esos casos.
- Si el tiempo de acceso a una palabra en cache es 1 ns, el tiempo de lectura o escritura de una palabra en memoria principal es 50 ns, el tiempo de transferencia de un bloque de MP a cache es 200 ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?
- Si se añade al computador una memoria cache de segundo nivel de 1MB y asociatividad 4, con bloques de 16 bytes, con asignación en escritura y actualización diferida y si el tiempo de transferencia de un bloque desde memoria cache de segundo nivel a la de primer nivel son 15ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?



Memoria Cache - Problemas adicionales

1.- Sea un computador con memoria cache con las características siguientes:

Memoria física de 32 KB

Memoria cache de direcciones físicas de 512 bytes con bloques de 128 bytes y emplazamiento directo

- Indique el formato de la dirección para MP teniendo en cuenta las características de la MC
- Si en un momento dado el directorio de la cache tiene los contenidos (en hexadecimal) indicados en la tabla, exprese en hexadecimal el rango de direcciones físicas ubicadas en cada bloque de la memoria cache.
- Suponiendo que un programa realiza la siguiente cadena de referencias indique el número de aciertos que se producen: de 0x2080 a 0x209F, de 0x2880 a 0x289F y de 0x03F0 a 0x0410.

Etiqueta	Bloque
35	0
10	1
10	2
08	3

2.- Sea una memoria principal de 32M bytes direccionable por bytes a la que se dota de una memoria cache con las siguientes características:

- Tamaño de 2K bytes
- Líneas de 256 bytes,
- Prebúsqueda bajo fallo (en caso de fallo se trae el bloque que lo provoca y el siguiente)
- Algoritmo de reemplazamiento FIFO.

Sea la secuencia de acceso a memoria principal dada por las siguientes direcciones: 0x0023FA, 0x0014A2, 0x003F02, 0x0040B1, 0x005572 y 0x0023AA. Muestre la evolución del directorio cache indicando los fallos (F), las prebúsquedas (P) y los aciertos (A) que se producen suponiendo:

- emplazamiento directo
- emplazamiento asociativo por conjuntos de 2 vías

3.- Se dispone de un computador con una memoria direccionable por bytes y dotado de una cache de 256 bytes. Cada bloque es de 16 bytes.

a) Si la cache es de organización directa, indique para la siguiente secuencia de referencias a memoria si se trata de un acierto o un fallo: 0xA01, 0xB1F, 0x70A, 0x60F, 0xA70, 0xB11, 0xA7A, 0xA0B, 0x67A, 0xA7F, 0x071, 0x67F. Indique además qué fallos producen reemplazamiento.

b) Si la cache es asociativa por conjuntos con dos vías, con reemplazamiento LRU, indique para la misma secuencia de referencias del apartado anterior si se producen reemplazamientos.

4.- Considere un computador con una memoria principal de 128MB, direccionable por bytes, al que se dota de una memoria cache de 1MB, con líneas de 256B, y cache de víctima de 1KB. La memoria cache tiene acceso directo, mientras que la cache víctima es completamente asociativa. Para ambas el algoritmo de reemplazamiento es FIFO. Originalmente en las dos memorias se encuentran los bloques que aparecen en las siguientes tablas.

Sea la secuencia de acceso a memoria principal dada por las siguientes direcciones: 0x334500, 0x14BF00, 0x15BF00, 0x14BFFF, 0x402100, 0x55AB00, 0x41F100y 0x3345FF.

Suponiendo la cache originalmente vacía muestre la evolución de la caché de etiquetas indicando los fallos y las transferencias entre cache de víctima y cache.

5.- Considere un computador con una memoria principal de 4MB, direccionable por bytes, al que se dota de una memoria cache de 2KB, con líneas de 512B. La memoria cache presenta emplazamiento directo. Se quiere ejecutar el siguiente código:

```
for(i=0; i<1024; i++) {
    C[i]=A[i]-B[1023-i];
}
```

Los datos son enteros y están almacenados desde la dirección 0x200000

- ¿Cuántos fallos de cache se producen?
- ¿Cuántos fallos se producen si aplicamos fusión de arrays? ¿Importa el orden de la fusión?
- ¿Cuántos fallos se producen si aplicamos alargamientos de arrays? ¿Importa la cantidad "alargada"?

6.- Considere un computador con direcciones de 32 bits, al que se dota de una memoria cache de 4KB, con líneas (bloques) de 16 bytes y emplazamiento directo.

Se quiere ejecutar el siguiente código:

```
int A[1024]; // A[0] están en la dirección 0x0C000000
int B[1024];
int C[1024];

for (i=0; i<10; i++) {
    for(j=0; j<1024; j++) {
        C[i] += A[j]*B[j];
    }
}
```

- ¿Cuántos fallos de cache se producen?
- ¿Qué asociatividad necesitamos (como mínimo) para evitar todos los fallos por conflicto con este código?
- Siguiendo con emplazamiento directo, ¿qué modificaciones de código podríamos hacer para evitar todos los fallos por conflicto?

7.- Considere el siguiente código que implementa una multiplicación de matrices:

```
int A[128][128];
int B[128][128];
int C[128][128];

for (i=0; i<128; i++) {
    for (j=0; j<128; j++) {
        C[i][j]=0;
        for (k=0; k<128; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

- ¿Qué matriz presenta peor localidad en este código? ¿Se puede mejorar mediante intercambio de bucles? Al mejorar la localidad de esa matriz, ¿empeora la de alguna otra?
- Busque información sobre la transformación llamada "loop tiling" (a veces "blocking" o bloqueo). Aplíquela a este código y discuta si disminuiría el número de fallo de cache (no es necesario cuantificar la mejora; sólo discutir si es una transformación conveniente).

8.- Se va a ejecutar el código que se muestra a continuación en una máquina con direcciones de 16 bits que dispone de una memoria cache de datos de acceso directo con 4 bloques de 16 bytes cada uno y política de escritura write-back con asignación en escritura. Teniendo en cuenta sólo la cache de datos (es decir, ignorando la lectura de instrucciones):

- Ignorando los accesos a la variable tmp (estará en registro), ¿cuántos fallos de cache se producirán? Justifique la respuesta, indicando además cuántos reemplazamientos se producen y en qué iteraciones.
- Recalcule el número de fallos si la cache fuese asociativa con 2 vías (2 bloques por conjunto) y la política de reemplazamiento fuese LRU.
- ¿Qué etiqueta (expresada en hexadecimal) tendrá el (marco de) bloque 2 de la cache tras la ejecución del código?

```
//Cada entero ocupa 4 bytes
int v[16]; // Dirección de comienzo 0x0000
int mask[4]; // Dirección de comienzo 0x0040

for(i=0; i<12; i++) {
    tmp = 0;
    for(j=0; j<4; j++){
        tmp = tmp + mask[j]*v[i];
    }
    v[i]=tmp;
}
```

Ejercicios Memoria Caché

①

$$MC = 32KB = 2^{15}$$

$$\text{bloques } 256 \text{ bytes} = 2^8 \text{ bytes}$$

#define N 128

int A [N][N]

int B [N][N]

int C [N][N]

for (i=0; i<N; i++)

for (j=0; j<N; j++)

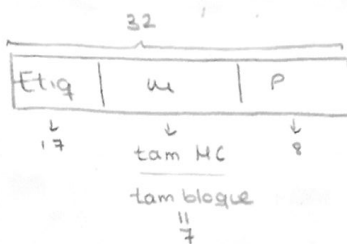
C[i][j] = A[i][j] + B[i][j];

Comienzo: $0x00000000$

$$8 \text{ dígitos} \times 4 = 32$$

porque está en hexadecimal

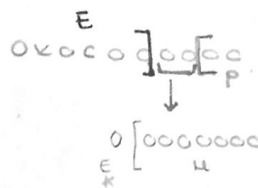
a)



A[0][0] → 0x00000000

A[0][1] → 0x00000004

⋮



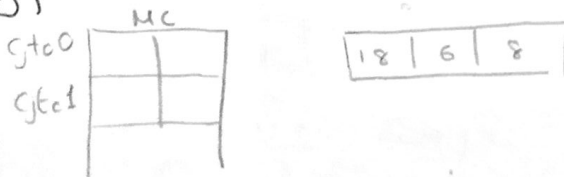
Bloque 0 → 01800

$$128 \times 128 \times 4 = (65536)_2 \rightarrow (10000)_6$$

B[0][0] → 0x00010000

C[0][0] → 0x00020000 → Si no está en la cache se escribe en mem princ.

b) Fallos: $128 \times 128 \times 2$



0x00000000 → 00000000

Etiqu 000011000

A[0][64] fallo

B[0][64] fallo

A[1][0] fallo

B[1][0] fallo

$$\text{Fallo} = \frac{128 \times 128}{64} \times 2$$

MC

Gto0	03000	03004
Gto1		

Etiqueta de C



Fallos

$$128 \times 128 \times 3$$

UP 64 x
CH 2x1
líneas
prebas

2)

c) #define N128

```
struct d  
  int A;  
  int B;
```

```
  {  
    ABCN3CN3  
    int CCN3CN3
```

```
  for (i=0 ; i<N ; i++)
```

```
    for (j=0 ; j<N ; j++)
```

```
      C[i][j] = A[i][j] + B[i][j]
```

No fallas

AB[6][32] . A

AB[0][64] . A

$$\text{Fallo} = \frac{128 \times 128}{32} + \frac{128 \times 128}{64}$$

- 2) HP 64 Kbytes $\rightarrow 2^{16}$ bytes
 CU 2 Kbytes $\rightarrow 2^{11}$ bytes
 líneas 256 bytes $\rightarrow 2^8$
 Prebúsqueda bajo fallo.

0x3345 0x14BF 0x4021 0x1184
 0x55AB 0x1284 0x41F1

a) Organización directa.



Bloque	Etiqueta
0	08
1	08 02 08
2	02
3	06
4	06 02
5	02 0A
6	0A
7	

1) 0x3345 $\rightarrow$ 0011001101000101
 06 3

Fallo

Siguiente: (Prebúsqueda)

00110100
 E M
 06 4

2) 0x14BF $\rightarrow$ 00010100
 E M
 02 4

Fallo

Siguiente

00010101
 02 5

5) 0x55AB

01010101
 0A 5

Fallo

6) 0x1284

00010101
 02 2

Acierto

3) 0x4021

Fallo

01000100
 E M
 08 0

Siguiente:

01000101
 1

4) 0x1184

Fallo

00010001
 02 1

Siguiente

00010101
 02 2

7) 0x41F1

Fallo

01000101
 08 1

b) Conjuntos de dos años, L2U

6	2	2
---	---	---

Cyto | Etiqueta

$$\left[\begin{array}{cccccccccccccccc} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{array} \right]$$
$$\begin{array}{r} 5000 \\ - 1000 \\ \hline 4000 \end{array}$$

2). Oxidation

$$\begin{array}{r} 1000 \\ 1000 \\ \hline 2000 \end{array}$$

815

$$\begin{array}{r} 100 \\ 100 \overline{) 10000} \\ \underline{10000} \\ 0 \end{array}$$

NOTES

Fall 9
$$\begin{array}{r} 00 \\ \hline 010000 \\ \hline 00 \end{array}$$

1 RLU, Sig. 010000001

5 4 3 2 1

$$\begin{array}{r} 100010001 \\ \hline \end{array}$$
$$\begin{array}{r} 8.5 \\ \hline 1001000 \end{array}$$

5). 0x55A0

$$\begin{array}{r} 15 \\ 9 \overline{) 135} \\ \underline{9} \\ 45 \\ \underline{45} \\ 0 \end{array}$$

$$\begin{array}{r} 257 \\ 2 \overline{) 514} \\ \underline{514} \\ 0 \end{array}$$

$$\pi \cdot 5 \cdot 41$$

5000

$$\frac{01000001}{10} = 12001$$

Sig. 0100010

6) $6 \times 128 \gamma$

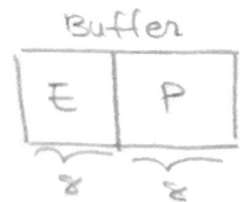
$$\begin{array}{r} 2000 \\ 2000 \\ \hline 4000 \end{array}$$

Aciento

c) org. directa

buffer 512 bytes.

EA
↓
buffer



Bloque	Etiqueta
0	08
1	02
2	02
3	06
4	02
5	00
6	
7	

Buffer

34	18
----	----

41 12
56 42

1) 0x3345

Fallo

00110011
06 3

2) 0x14BF

00010100 Fallo
02 4

Buffer → 15
el sig

1 FIFO

00010100
4 5

3) 0x4021

01000000
02 0

Fallo

Sig → 01000001
4 1

4) 0x1184

00010001 Fallo
02 1

Sig → Buffer 00010010
1 2

5) 0x55AB

01010101 Fallo
04 5

Sig → Buffer

01010110
5 6

6) 0x1284

00010010
02 2

00010010
1 2

7) 0x41F1

01000001
02 1

Fallo

Sig → Buffer

01000010
1 2

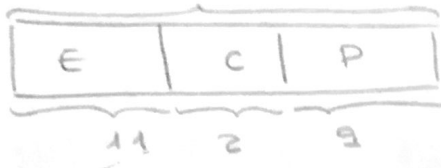
Acierto del buffer
a cache

UP 4MB = 2^{22}

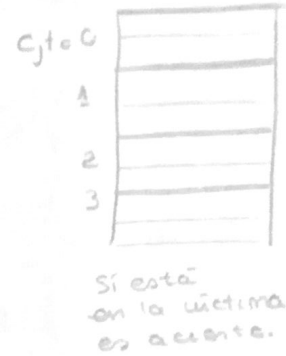
HC 4KB con línea 512 bytes.

CV 1024 bytes asociativa.

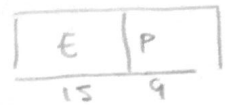
HC 2 vías.
22



Cache víctima



Cuando se quita, no desaparece, se guarda en la cache víctima



③ HC

22		
Etig	C	P
11	2	9

CV

Etig	P
13	9

CV:

Etig	FIFO
CA80	810
19A0	
3FF	101
0020	

HC:

Gte	Etig	FIFO
0	668 008	810 1
	008 2A0	810 1
1	297	0
	368	1
2	5FF	0
	688	1
	297	1
3	200	0

1ª) 0x334500

con 2 bits, los demas con 1

11 00 11 01 00 01 01 00000000
E C P
668 2

Acierto en MC

2ª) 0x14BF00

0 1 0 1 0 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0
E C P

Acierto en MC

3ª) 0x150084

0 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0
E C P
2A0 0
et. HV
0A80

Fallo MC.

Acierto CV → lo llevamos de HV a MC

008 → 0000000010001000 ^{eto} ⇒ 0020
dos más por el etig
HV es 13 bits.

5ª) 0x0540AB

00000000 01 01 00 00 00 10 10 10 11
E C P
Fallo MC
Fallo CV

00010101000000
0 2 A 0

4ª) 0x004021

00000000
008 2A0 C P

Fallo MC.

Acierto CV

0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0A80

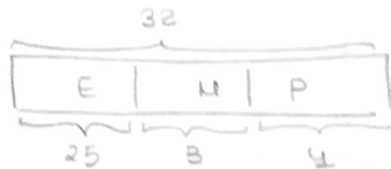
11 00 11 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 9 A 0 (668)

6ª) 0x0041F1

00000000
008 E C P

Acierto MC

④ a) fallos de cache.



BLOQUE	ETIQUETA
0	
1	media[4...7]
2	media[8...11]
3	media[12...15]
4	
5	
6	
7	

$$128 \times 4 = (2^9) \Rightarrow (200)_{10}$$

↑
tam por

int nota[128] 0x00000000

int media[128] 0x00000200

$$7 < i < 64$$

media[8] fallo MC

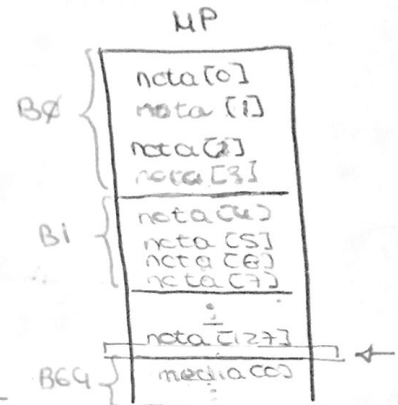
$$\text{bloques } 16 \text{ bytes} = 2^4 \Rightarrow 4 \text{ PAL}$$

MC 128 bytes

$$\frac{128 \text{ bytes}}{16 \text{ bytes}} = \frac{2^7}{2^4} = 2^3 = 8 \text{ bloques}$$

nota[0] → 0x00000000
Fallo
media[0] → 0x00000200
Fallo

nota[1]
Fallo
media[1]
Fallo



$0 \leq i \leq 7$
Fallos en $0 \leq i \leq 7$
↓
16 fallos.

nota[i] = media[i]/2 → nota[8] fallo escritura ⇒ escrib. MP
...
nota[11]

media[12] fallo MC

fallo escritura ⇒ nota[12]

$$\Rightarrow 63 - 8 + 1 = 56 \text{ veces.}$$

nota[15]

$$\frac{56}{4} = 14 \text{ fallos MC.}$$

$$64 \leq i < 128$$

nota[64] → Fallo MC

media[64] → Fallo MC

nota[65] → fallo MC

media[65] → fallo MC

(igual que $0 \leq i < 7$)

$$127 - 64 + 1 = 64$$

nº fallos → $2 \times 64 = 128$ MC

$$\boxed{\text{Total fallos MC} = 158}$$

$$\boxed{\text{Total aciertos} = 42}$$

$$\boxed{\text{Total fallos MP} = 56}$$

c) MC 4 ns

Escrib en MP 50 ns

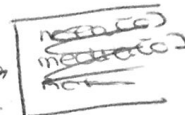
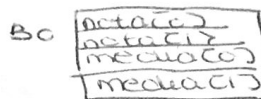
Transferencia MP → MC 200 ns
+ bloque.

Calcular tiempo:

$$T_{\text{total}} = 128 \times 200 \text{ ns} + 36 \times 50 \text{ ns} + 42 \times 1 \text{ ns} + 128 \times 1 \text{ ns} = 34.600 \text{ ns}$$

Solución: 1) dejar un bloque vacío en medio

2) Cada bloque tener 2 de uno y 2 de otro.

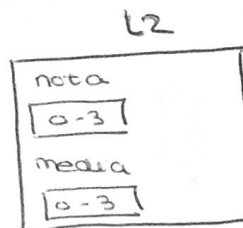
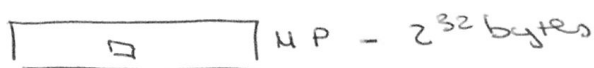
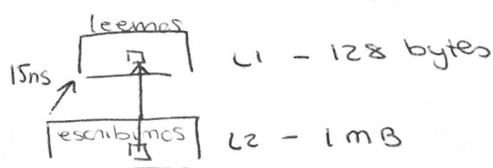


d) Cache de segundo nivel.

MC L2 1 MB asociatividad 4. (cada conjunto 4 bloques)

Bloques 16 bytes

Asignación escritura.



$i \leq 7$

n[0] fallo MP
m[0] fallo MP
n[1] fallo L2
m[1] fallo L2

$128 > i \geq 64$

Se van reemplazando como en el otro.

$7 < i < 64$

media[8] fallo MP
nota[8] → L2
media[9] → Acierto
nota[9] → Acierto L2
Fallo MC.

nota[11] igual ↑

media[12] nota[12] { fallos ...

Penalización por fallos

$$(200 + 200 + 3 \times 15 \times 2) \times 2 + 16 = 996 \text{ ns}$$

↑ ↑ ↑
tras de L2 2 veces 8 pun de n
8 pun de m

$$32 \times (200 + 200 + 3 \times 2 \times 15) + 2 \times 128 = 15936 \text{ ns}$$

$$\frac{128}{4} = 32 \text{ veces}$$

~~128~~ $128 > i \geq 64$

$$= 15936 \text{ ns}$$

Falta dato!

Se supone igual.

WUOLAH